

RTRL Flight Controller — Live Demo Guide

How to run `scripts/live_demo.py`, what it is, what the graphs show, and what to expect from each trigger.

1. What the live demo is

The live demo is an interactive, ad-hoc dashboard — it is **not** part of the training pipeline and does not write any parquet trace files or W&B logs. It runs a single continuous episode of the Cessna 172 attitude-hold task in real time, using whichever controller and environment scenario you pick from the command line, and plots the aircraft's behavior live as it happens.

Its purpose is to let you **see** the project's central claim happen on screen: that the RTRL-RTU controller keeps adapting its weights mid-flight and recovers from a sudden actuator fault, while a frozen BPTT-LSTM (trained offline, weights fixed at deploy time) does not.

2. How to run it

Run from the repository root using `uv`, which resolves the project's managed virtual environment so the `rtrl_flight` package is importable (a plain python `scripts/live_demo.py` will fail with `ModuleNotFoundError: No module named 'rtrl_flight'`):

```
uv run python scripts/live_demo.py --controller <controller> --env <env>
```

--controller — one of:

- `pid` — classical PID baseline, fixed gains
- `bptt_lstm` — LSTM trained offline via BPTT, weights frozen at deploy time
- `rtrl_rtu` — the RTRL controller; keeps updating its weights online, every step, via forward sensitivity

--env — one of:

- `cessna172_nominal` — calm air, no faults; f/e/r/w buttons are no-ops here
- `cessna172_fault` — supports fault injection (f/e/r), no wind wrapper
- `cessna172_wind` — supports wind cycling (w), no fault wrapper
- `cessna172_combined` — both fault injection and wind cycling available — the headline demo

Recommended first run, since it shows the project's core claim most directly:

```
uv run python scripts/live_demo.py --controller rtrl_rtu --env cessna172_fault
```

A matplotlib window opens showing four live-updating panels plus a row of buttons along the bottom. Nothing is written to disk — close the window (or Ctrl+C in the terminal) to end the session.

3. What the four panels show

Panel	What it plots	How to read it
Attitude tracking (rad)	Pitch & roll (solid lines) vs. their commanded targets (dashed lines), last 200 steps.	The tighter the solid line hugs its dashed target, the better the controller is holding the commanded attitude. Growing gaps mean the aircraft is drifting off target.
Control surfaces	The three commanded actions — aileron, elevator, rudder — each step.	Watch for sudden jumps (a fault just hit) and for whether the signal is smooth or jittery (jitter = one of this project's control-smoothness metrics).
RTRL diagnostics	Smoothed online loss (blue, left axis) and the sensitivity Frobenius norm $ S _F$ (orange, right axis).	Only <code>rtrl_rtu</code> populates these meaningfully — PID and BPTT-LSTM have no online update, so both lines sit flat at zero for them. For RTRL, a loss spike right after a fault followed by a decay back down is the online weight update "learning" the new (degraded) dynamics in real time. The sensitivity norm is a health indicator for the RTRL math itself — it should stay bounded, not blow up.
Status text	Controller name, active fault state, wind level, step count, running episode return.	The at-a-glance summary of what's currently being injected into the episode.

4. Buttons / triggers — what to expect

A row of five buttons sits along the bottom of the window (each also bound to a keyboard shortcut — `f / e / r / w / space` — if the window has keyboard focus). Fault and wind buttons print a console message and do nothing if the chosen `--env` has no matching wrapper (e.g. faults on `cessna172_nominal`).

Button	What it does	What to expect on screen
Fault: aileron	Degrades aileron authority to 50% (<code>effective_action = commanded × 0.5</code>) starting next step.	Roll tracking degrades — the roll line drifts off its dashed target and the aileron trace looks "weaker" for the same commanded input. With <code>rtrl_rtu</code> , expect a brief online-loss spike, then roll tracking gradually tightening back up over the next several seconds as the controller adapts. With <code>pid</code> or <code>bptt_lstm</code> , the degraded tracking persists — neither adapts its policy at runtime.
Fault: elevator	Same idea, applied to the elevator (pitch axis) at 50% severity.	Same pattern, on pitch instead of roll: pitch drifts off target, RTRL recovers over time, PID/BPTT-LSTM stay degraded.

Button	What it does	What to expect on screen
Fault: rudder	Same idea, applied to the rudder at 50% severity.	Rudder mainly affects yaw/coordination rather than the plotted pitch/roll targets directly, so its effect is subtler on the Attitude panel — watch the Control surfaces panel's rudder trace and the status text's Fault line instead to confirm it's active.
Cycle wind	Advances wind severity: off → light → moderate → severe → back to off.	Higher severities inject stronger Dryden/Tustin turbulence, which shows up as noisier pitch/roll traces and more corrective jitter in the Control surfaces panel. This is ambient disturbance, not an actuator failure — even PID should mostly keep tracking through it, just noisily; it's meant to stress-test smoothness, not recovery.
Reset episode	Ends the current episode and starts a new one. Per the Controller ABC, <code>reset ()</code> only clears episode-local state (hidden state, sensitivity tensors, etc.) — it does not revert any online weight updates RTRL has made.	All four panels clear and start accumulating from step 0 again. Fault severities also reset to full authority (100%) for the new episode; wind severity is <i>not</i> reset — it's treated as an ambient run-level setting and carries over.

5. Suggested demo sequence

- 1 Run `--controller rtrl_rtu --env cessna172_fault` and let it settle for ~5–10 seconds of clean tracking.
- 2 Click **Fault: aileron** (or elevator). Watch the Attitude panel drift, the RTRL diagnostics online-loss spike, then tracking gradually tighten back up as the sensitivity norm stays bounded.
- 3 Click **Reset episode**, then re-run the exact same steps with `--controller bptt_lstm` on the same env. The same fault degrades tracking the same way, but it never recovers — the frozen weights can't adapt.
- 4 This side-by-side (RTRL recovers / BPTT-LSTM doesn't) is the project's headline result, per `CLAUDE.md`'s success criterion.

Note: this guide describes `scripts/live_demo.py` as of the version with button-triggered controls (buttons + f/e/r/w/space keyboard shortcuts, both wired to the same underlying trigger functions).